

CSCE 645 GEOMETRIC MODELING · MODULE 01 · LECTURE NOTES

Representing Shape

Four ways to describe a surface, and what each one makes easy.

MODULE OUTCOMES

Distinguish parametric, implicit, explicit, and volumetric representations.

Predict which operations are cheap and which are hard in each.

Explain why conversion between representations is lossy.

Choose a representation from the operations a task requires.

State what a manifold is and why the condition matters.

1 Four ways to describe a surface

Before any algorithm, a decision: how is the shape stored? The answer determines which questions are one line of code and which are a research project.

Family	Definition	Examples	Natural operation
Parametric	A function from a parameter domain to space: $S(u,v) \rightarrow \mathbb{R}^3$.	Bézier and NURBS patches; a sphere by latitude and longitude.	Generate points on the surface.
Implicit	The zero set of a function: $\{x : f(x) = 0\}$.	Signed distance fields, metaballs, algebraic surfaces.	Test whether a point is inside.
Explicit / discrete	An enumerated collection of primitives.	Triangle meshes, point clouds.	Render; edit locally.
Volumetric	A scalar field sampled on a grid.	Voxel grids, CT and MRI data, level sets.	Represent interiors and topology change.

1.1 The fundamental asymmetry

EACH REPRESENTATION MAKES ONE QUESTION TRIVIAL AND ANOTHER HARD

'Give me a point on the surface.' Parametric: evaluate $S(u,v)$. Trivial. Implicit: you must solve $f(x) = 0$, which means root-finding along a ray.

'Is this point inside?' Implicit: evaluate f and check the sign. Trivial. Parametric or mesh: cast a ray and count crossings, with all the degenerate cases that implies.

'Union these two shapes.' Implicit: $\min(f_1, f_2)$. One line. Mesh: compute every intersection curve, re-triangulate both surfaces along it, and resolve numerical near-misses — a genuinely difficult problem that commercial kernels still get wrong.

The skill is not mastering one representation. It is recognising which makes your actual question trivial, and knowing what conversion costs.

2 Why triangle meshes dominate

Meshes are not the most expressive or the most mathematically convenient representation. They dominate for an engineering reason: graphics hardware rasterises triangles, and everything else must be converted to triangles before it can be drawn.

Strength	Weakness
Native to the rendering hardware (CSCE 641).	No exact smoothness: curvature is zero on every face and undefined on every edge. Module 07 is about repairing this.
Arbitrary topology at no cost — any genus, no patch layout.	Boolean operations are difficult and numerically fragile.
Local editing: move one vertex, nothing else changes.	Resolution is baked in. Refining requires inventing detail that is not in the data (Module 06).
Adaptive: small triangles where detail is needed.	No compact analytic description — storage grows with detail.

3 The operation cost table

Operation	Parametric	Implicit	Triangle mesh
Evaluate a point on the surface	Trivial	Root-find along a ray	Trivial
Inside/outside test	Hard	Sign of f	Ray cast, count crossings
Boolean operations	Very hard	min / max of fields	Fragile; a real engineering problem
Render	Tessellate first	Ray march or extract a mesh	Native

Operation	Parametric	Implicit	Triangle mesh
Exact normal	Cross product of partials	∇f , normalised	Averaged from faces — approximate
Arbitrary topology	Awkward — needs trimming and patch layout	Free	Free
Local edit	Affects a whole patch	Hard to localise	Move one vertex
Offset by a distance	Hard	$f(\mathbf{x}) - d$	Self-intersects

Keep this table. Almost every design decision in a geometry pipeline is a row of it, and most pipeline complexity comes from needing operations that live in different columns.

4 Manifolds

Most algorithms in this course assume the mesh is **manifold**. The condition sounds abstract and has a concrete combinatorial meaning.

A surface is manifold if every point has a neighbourhood homeomorphic to a disc — locally, it looks flat. For a triangle mesh this reduces to two checkable conditions:

- 1 **Every edge is shared by exactly two faces** (or by one, if it is a boundary edge). An edge with three incident faces is a non-manifold edge.
- 2 **The faces around every vertex form a single closed fan**. Two cones meeting at a single shared vertex is non-manifold, even though every edge is fine.

REAL MESHES VIOLATE THIS CONSTANTLY

Scanned data has holes, duplicated vertices, and self-intersections. Authored data has T-junctions, zero-area triangles, and geometry intentionally built from intersecting parts. Exported data has duplicated vertices wherever a UV or normal seam occurs.

A library that assumes manifold input and gets none will fail somewhere deep in a traversal, often silently producing wrong results rather than crashing.

Validate on load and report the specific violation. This is required in Project 1, and it is the difference between a library you can trust and one you cannot.

4.1 Euler's formula

$$V - E + F = 2 - 2g$$

for a closed, connected, manifold surface of genus g — where the genus counts handles. A sphere has $g = 0$ so the alternating sum is 2; a torus has $g = 1$ so it is 0; a two-handled surface gives -2 .

This is a **topological invariant**. Bending, stretching, and subdividing the mesh leave it unchanged; only cutting or gluing alters it. That makes it an excellent and extremely cheap integrity check: compute $V - E + F$ on any mesh you load, and if it does not match the expected genus you have holes, duplicated vertices, or non-manifold elements — discovered immediately rather than three algorithms later.

4.2 A corollary worth memorising

In a closed triangle mesh every face has three edges and every edge is shared by two faces, so $3F = 2E$. Substituting into Euler's formula and taking V large:

$$F \approx 2V \quad E \approx 3V \quad \text{average valence} \approx 6$$

WHY VALENCE 6 KEEPS REAPPEARING

The average vertex of a closed triangle mesh has six neighbours, regardless of the shape. This is forced by topology, not by any choice of the modeller.

It is why a well-made triangle mesh looks locally like a hexagonal packing, why remeshing algorithms (Module 10) explicitly drive vertex valence toward 6, and why vertices of other valence are called **irregular** or **extraordinary** — a term that returns in Module 06, where subdivision surfaces lose a degree of smoothness precisely at those points.

The analogous number for quad meshes is 4, which is why quad modelling guidelines are so insistent about avoiding poles.

5 Choosing

Task	Representation	Why
Rendering a character or environment	Triangle mesh	The hardware requires it, and everything converts to it eventually.
CAD with exact booleans and offsets	NURBS B-rep, or implicit	Exactness and well-defined set operations matter more than rendering speed.
Procedural terrain, clouds, or effects	Implicit	Evaluate anywhere at any resolution; no storage proportional to detail.
Medical imaging, simulation output	Volumetric	It is how the data was acquired, and interiors matter.
Smooth authored organic surfaces	Subdivision (Module 06)	A coarse editable cage with a smooth limit surface, converting to mesh on demand.
Collision detection	Implicit or BVH over a mesh	Distance queries are what implicits are for.

Production pipelines use several representations and convert between them. The conversions are where information is lost and where most pipeline bugs live: a NURBS surface tessellated to a mesh has lost its exact parameterisation; a mesh converted to a distance field has lost its sharp edges unless they were specially handled. Knowing what each conversion discards is a large part of working with geometry professionally.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Keenan Crane — DDG, Lecture 1 (Overview) and the Combinatorial Surfaces lecture	Representations, manifoldness, and Euler characteristic, with the topology done carefully.
libigl tutorial — Chapter 1 (Mesh representation)	How a real library stores meshes, and why it uses flat index matrices rather than pointer structures.
Inigo Quilez — 'Distance functions'	The implicit side, with working code. Read it now for contrast; it becomes the primary source in Module 11.
Polygon Mesh Processing slides — Chapter 1	Representations compared with the operation-cost framing used here.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Load several meshes and compute V , E , F , and the Euler characteristic for each. Verify a sphere gives 2 and a torus gives 0. Find or construct a mesh where it gives something unexpected and diagnose why.
- 2 Write a validator that detects non-manifold edges (three or more incident faces), non-manifold vertices (more than one face fan), isolated vertices, and duplicate vertices. Run it on a scanned model and report what you find.
- 3 Compute the average vertex valence for five different triangle meshes and confirm it approaches 6. Explain any deviation in terms of boundary vertices.
- 4 Implement a sphere three ways: parametric by (θ, ϕ) , implicit as $|x|-r$, and as a triangle mesh. For each, write the code to answer 'is this point inside?' and 'give me a point on the surface'. Compare the effort.
- 5 Implement the union of two spheres implicitly with $\min(f_1, f_2)$, and then attempt the same operation on two sphere meshes. Do not finish the mesh version — the point is to discover how much work it is.
- 6 Take a NURBS or analytic surface, tessellate it at two resolutions, and describe precisely what information the conversion discarded.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four representation families and the operation each makes trivial.
- 2 Why are boolean operations trivial for implicit surfaces and hard for meshes?
- 3 Give the real reason triangle meshes dominate graphics, and two things they are genuinely bad at.

- 4 State the two combinatorial conditions for a triangle mesh to be manifold.
- 5 What is Euler's formula, why is it invariant, and what practical use does it have?
- 6 Derive average valence ≈ 6 for a closed triangle mesh, and say why the number matters later.